MatShrink: Lossless Weight Compression for Back-to-Back Linear Layers in Transformers

Nils Graef[*], **Filip Makraduli**, **Siddh Shah**, **Siddharth Mohan**
OpenMachine

Abstract

We present MatShrink, a mathematically *exact* method for reducing the parameter count of any pair of consecutive linear layers. Unlike approximate compression techniques such as SVD-based rank reduction or magnitude pruning, MatShrink eliminates an $r \times r$ block of weights via an invertible change of basis on the shared inner dimension r, incurring zero accuracy loss by construction. We state and prove the core identity as a formal proposition and establish conditions under which the transformation is numerically stable. We specialize MatShrink to the V-O, Q-K, and latent projections of multi-head attention (MHA), multi-head latent attention (MLA), grouped-query attention (GQA), and multi-query attention (MQA), yielding lossless savings of 8–17% of per-layer attention parameters across representative models. We identify a bf16 numerical instability caused by ill-conditioned transformation matrices and show that pivoted QR basis selection improves numerical stability. Empirical validation on four models (GPT-2-124M, SmolLM2-135M, Gemma-3-1B, Gemma-3-4B) confirms exact fp32 equivalence and near-exact bf16 equivalence, with checkpoints loading directly in HuggingFace Transformers and vLLM without any runtime modification. MatShrink is applicable to both inference and training: Existing models can be retrofitted with MatShrink in a mathematically equivalent way, and future models can use MatShrink for both training and inference. See [1] for code and more transformer tricks.

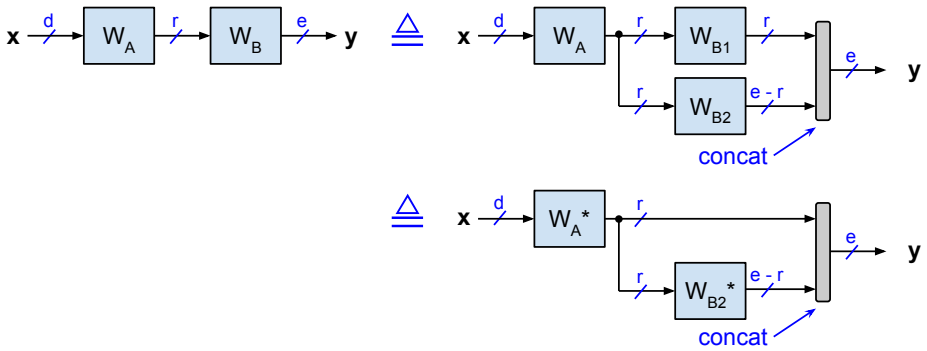

Figure 1: Mathematically equivalent implementations of two back-to-back weight matrices W_A and W_B (left) with rank r, where $d > r$ and $e > r$, which reduces the total number of weights by r^2 (right).

[*] info@openmachine.ai

1 Introduction

The dominant cost of serving large transformer models is parameter count: it dictates memory footprint, memory bandwidth, and arithmetic throughput. A large body of work addresses this through *approximate* compression—SVD-based low-rank factorization [2, 3], magnitude or structured pruning, and quantization—all of which trade accuracy for efficiency. A complementary and largely underexplored direction is *exact* compression: identifying algebraic redundancies in the weight structure that can be eliminated without changing the function computed.

We observe that back-to-back linear layers—pairs of weight matrices applied sequentially with no nonlinearity between them—are ubiquitous in transformers and admit precisely such a redundancy. When $W_B \in \mathbb{R}^{r\times e}$ has full row rank r, any invertible $r \times r$ sub-block of W_B can be absorbed into W_A via an invertible change of basis on the shared inner dimension r, eliminating r^2 parameters while leaving the matrix product $W_A W_B$ unchanged. We call this procedure **MatShrink**.

MatShrink is complementary to approximate compression: it applies as a lossless post-processing step after SVD-based rank reduction, and is equally applicable to retrofitting existing models and to designing future architectures from scratch.

Contributions.

1. We state and prove MatShrink as a formal proposition and derive the closed-form weight savings (Section 2).
2. We show that the attention dot-product is preserved under MatShrink applied to Q-K projections, even though the individual queries and keys change (Section 3).
3. We specialize MatShrink to MHA, MLA, GQA, and MQA, quantifying per-model savings and establishing the correct application order for MLA (Sections 3–4).
4. We identify the bf16 numerical instability, diagnose its root cause as ill-conditioned transformation matrices, and show that pivoted QR basis selection improves the numerical stability (Section 7).
5. We empirically validate all theoretical claims on four models across two attention variants (Section 8).

Related work. Approximate weight compression schemes [2, 3] use SVD or other low-rank approximations to reduce parameter counts at the cost of accuracy; MatShrink is exact and incurs no accuracy loss. Slim attention [4] is the closest prior work, also using matrix inversion on V-O projections, but targeting *KV-cache* size rather than static weight count. Weight fusion for the special case $e \leq r$ (absorbing two matrices into one) appears in [5, 6]; MatShrink strictly generalizes this to $e > r$, where full fusion is infeasible but r^2 weights can still be removed losslessly. ConvBN fusion [7] is the convolutional analogue of MatShrink. Quantization-aware design [8] and weight sharing are orthogonal axes of compression that compose directly with MatShrink.

2 MatShrink: Formal Treatment

2.1 Setup and Core Proposition

Definition 1 (Back-to-back linear layer pair)**.** *Let $W_A \in \mathbb{R}^{d\times r}$ and $W_B \in \mathbb{R}^{r\times e}$ with $d > r$ and $e > r$. A* back-to-back linear layer pair *is the matrix product $W_A W_B$ with no nonlinearity between W_A and W_B.*

Proposition 2 (MatShrink)**.** *Let $W_A \in \mathbb{R}^{d\times r}$ and $W_B \in \mathbb{R}^{r\times e}$ with $\operatorname{rank}(W_B) = r$. Partition $W_B = [W_{B1}, W_{B2}]$ where $W_{B1} \in \mathbb{R}^{r\times r}$ is invertible and $W_{B2} \in \mathbb{R}^{r\times(e-r)}$. Define $W_A^* = W_A W_{B1}$ and $W_{B2}^* = W_{B1}^{-1} W_{B2}$, then:*

$$W_A W_B \;=\; W_A^* \big[I_r,\; W_{B2}^*\big], \tag{1}$$

where I_r is the $r \times r$ identity matrix. The right-hand side stores $dr + r(e - r) = dr + re - r^2$ parameters, a reduction of r^2 over the original $dr + re$.

Proof.

$$
\begin{aligned}
W_A^*[I_r,\, W_{B2}^*] &= (W_A W_{B1})[I_r,\, W_{B1}^{-1} W_{B2}] \\
&= W_A\big[W_{B1} I_r,\; W_{B1} W_{B1}^{-1} W_{B2}\big] \\
&= W_A[W_{B1},\, W_{B2}] = W_A W_B. \qquad \square
\end{aligned}
$$

Figure 1 illustrates Proposition 2 for a single pair.

Remark 3 (Dual formulation)**.** *An equivalent r^2 reduction is obtained by shrinking W_A instead. Partition $W_A = [W_{A1}; W_{A2}]$ with $W_{A1} \in \mathbb{R}^{r\times r}$ invertible, $W_{A2} \in \mathbb{R}^{(d-r)\times r}$. Then $W_A W_B = [W_{A1}; W_{A2}]\, W_B = [I_r; W_{A2}^*]\, W_B^*$ where $W_B^* = W_{A1} W_B$ and $W_{A2}^* = W_{A2} W_{A1}^{-1}$, see Fig. 2. This is the natural choice when it is preferable to preserve the structure of W_B.*

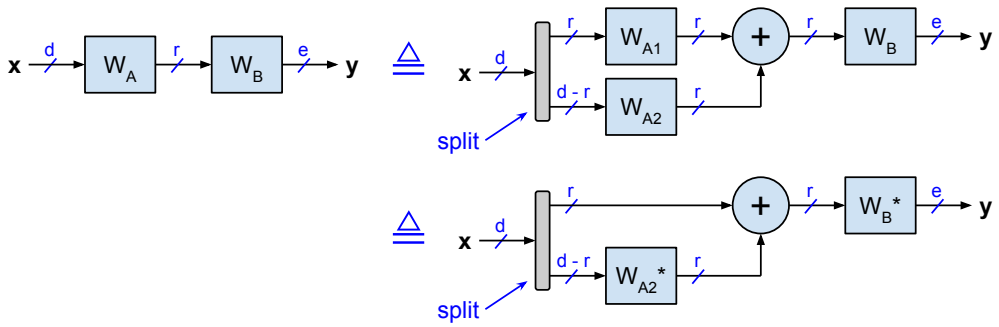

Figure 2: Alternative formulation (Remark 3): r^2 weights are eliminated from W_A instead of W_B.

For completeness, if $e = r$, then the entire matrix W_{B2} is eliminated. In general, if $e \le r$, then the two matrices W_A and W_B can be fused into a single matrix $W^* = W_A \cdot W_B$ with only de weights (instead of $dr + re$ weights for the original matrices W_A and W_B). This type of weight fusion is utilized in [5, 6].

2.2 Algorithm

Algorithm 1 is a complete, self-contained preprocessing procedure for retrofitting a pretrained model. Steps 1–4 are a one-time, per-layer cost; the forward pass (Step 5) requires no additional runtime modifications.

Algorithm 1 MatShrink: preprocessing a back-to-back layer pair

Require: $W_A \in \mathbb{R}^{d\times r}$, $W_B \in \mathbb{R}^{r\times e}$ with $e > r$
Ensure: Compressed pair (W_A^*, W_{B2}^*, P) computing the same function as (W_A, W_B)

1: **Pivot selection**: $(Q, R, P) \leftarrow \mathrm{pQR}(W_B^\top)$; set $M \leftarrow W_B[:, P[:r]]$ ▷ Section 7
2: **Inversion**: $M^{-1} \leftarrow$ fp64 LU factorization of M
3: **Absorb**: $W_A^* \leftarrow W_A M$ ▷ one-time $O(dr^2)$ cost
4: **Reduce**: $W_{B2}^* \leftarrow M^{-1} W_B[:, P[r:]]$ ▷ eliminates $r \times r$ block M
5: **Forward** (inference): $h \leftarrow x W_A^*$; output$[:, P[:r]] \leftarrow h$ (identity passthrough); output$[:, P[r:]] \leftarrow h \cdot W_{B2}^*$

3 MatShrink for MHA Transformers

V-O projections. In multi-head attention (MHA) [9], the value (V) and output (O) projections for head i form a back-to-back pair $W_{V,i} \in \mathbb{R}^{d\times r}$, $W_{O,i} \in \mathbb{R}^{r\times d}$, with no nonlinearity between them (Fig. 3). Proposition 2 applies directly.

MHA V-O savings. For MHA with h heads and model dimension $d = d_{\text{model}}$, each head has dimension $r = d_k = d/h$. Applying MatShrink to every head's V-O pair eliminates $r^2 = d^2/h^2$

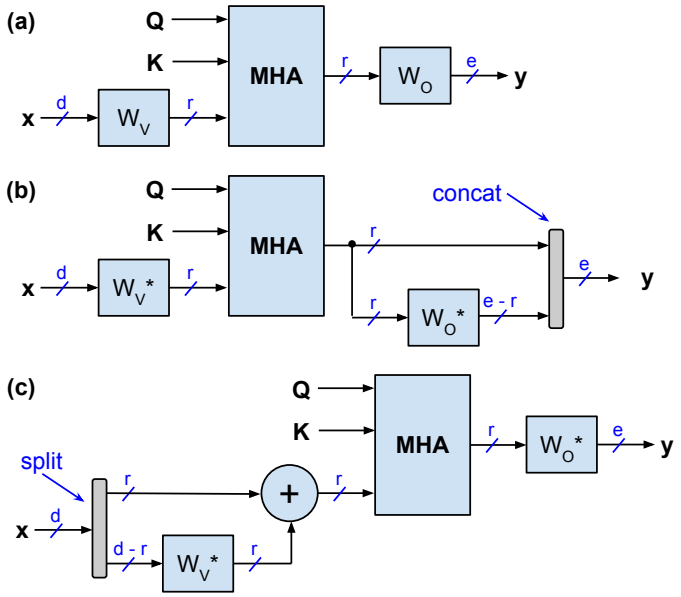

Figure 3: MatShrink (Proposition 2) applied to the V-O projection pair of a single attention head: (a) original; (b) compressed, eliminating $r^2 = d_k^2$ weights from W_O; (c) alternative formulation where r^2 weights are eliminated from W_V rather than W_o.

weights per head and d^2/h weights per layer in total—a fraction $1/(2h)$ of the $2d^2$ total V-O parameters. Savings per head $= r^2 = d^2/h^2$; summing over h heads gives d^2/h. Table 1 shows the MHA V-O savings for representative models.

Table 1: MHA weight savings from MatShrink (V-O projections). "Savings" is weights eliminated per layer; "savings %" is relative to all V-O parameters ($2d^2$ per layer). The savings formula $d_k^2 h$ follows from the MHA V-O savings derivation in Section 3.

Model	d	d_k	h	weights $d \times (d_k h)$	savings $d_k^2 h$	savings %
Whisper-tiny [10]	384	64	6	147K	25K	17%
CodeGemma-7B [11]	3,072	256	16	12.6M	1.0M	8%
T5-3B [12]	1,024	128	32	4.2M	0.5M	12%
T5-11B [12]	1,024	128	128	16.8M	2.1M	13%

Q-K projections. For models without RoPE and without QK-normalization (e.g., Whisper [10], T5 [12]), the query and key projections $W_{Q,i}, W_{K,i}$ form a back-to-back pair whose composition determines the attention logits.

Q-K dot-product invariance under MatShrink. MatShrink also applies to the query and key weight matrices W_Q and W_K, with one twist: unlike V-O, the goal is not to compress a sequential product $W_Q W_K$, but to preserve the attention dot-product $QK^\top = (xW_Q)(xW_K)^\top$. The key observation is that multiplying W_Q by any invertible matrix T and W_K by $(T^{-1})^\top$ leaves every dot-product unchanged, since the two transformations cancel:

$$(xW_Q T)\left(xW_K (T^{-1})^\top\right)^\top = xW_Q T T^{-1} (xW_K)^\top = (xW_Q)(xW_K)^\top.$$

Crucially, the transformed queries $Q^* = xW_Q T$ and keys $K^* = xW_K (T^{-1})^\top$ are individually different from the originals—yet their dot-product $Q^* {K^*}^\top$ is identical to $QK^\top$ for every possible input. This means the attention scores, and therefore the model outputs, are completely unchanged.

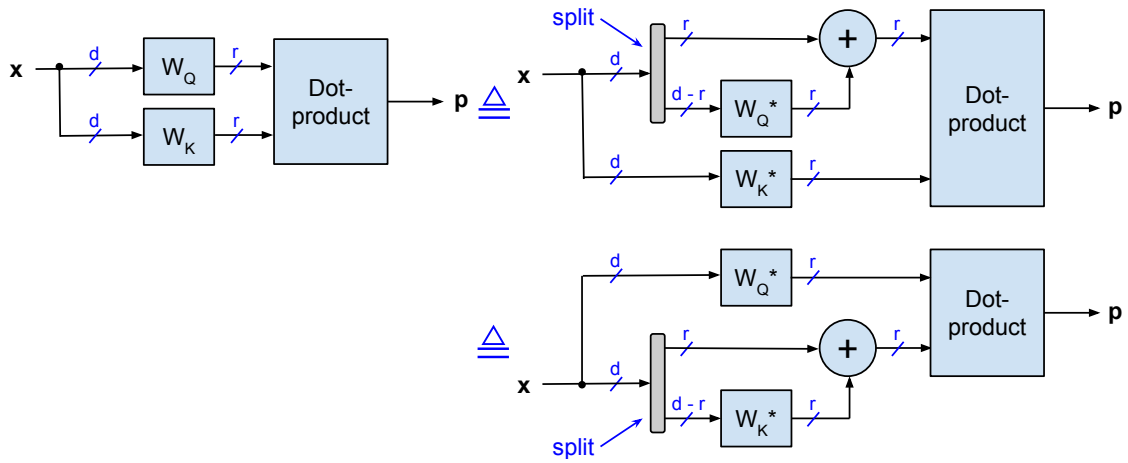

Figure 4: MatShrink for Q-K projections (Section 3) of a single attention head. Left: original Q and K projections and their dot-product p. Top right: equivalent implementation using MatShrink, which eliminates r^2 weights from W_Q. Bottom right: alternative implementation, which eliminates r^2 weights from W_K.

Partial RoPE. Many modern models apply RoPE to only a fraction $\alpha \in (0, 1)$ of d_k (typically $\alpha = 1/2$ or less). The dot-product invariance result above applies to the non-RoPE portion of size $(1 - \alpha)d_k$, yielding per-head savings of $(1 - \alpha)^2 d_k^2$.

4 MatShrink for GQA and MQA

GQA and MQA. For grouped-query attention [13] with grouping factor $g = n_{\text{heads}}/n_{\text{KV-heads}}$, MatShrink applies per KV-group. The per-group V-O savings are d_k^2 (from the MHA savings derivation above); the savings per query head are reduced by factor g relative to MHA. For MQA ($g = n_{\text{heads}}$), the single shared V-O pair still yields d_k^2 savings in absolute terms.

5 MatShrink for MLA Transformers

We follow the notation of DeepSeek-AI et al. [14]. r_{Q}: Q-latent rank; W_{DQ}: Q down-projection; W_{UQ}: Q NoPE up-projection; W_{QR}: Q RoPE up-projection; r_{KV}: KV-latent rank; W_{DKV}: KV down-projection; W_{UK}, W_{UV}: K and V NoPE up-projections; W_{KR}: K RoPE projection. Table 2 gives configurations of representative MLA models.

Table 2: Configurations of representative MLA transformer models.

Model	Params	d	r_{Q}	r_{KV}	h	d_{NOPE}	d_{ROPE}
Perplexity R1-1776, DeepSeek-R1, V3	685B	7,168	1,536	512	128	128	64
Mistral Large 3 675B	675B	7,168	1,536	512	128	128	64
DeepSeek-V2.5	236B	5,120	1,536	512	128	128	64
DeepSeek-V2-lite, VL2-small [14]	16B	2,048	N/A	512	16	128	64
OpenBMB MiniCPM3-4B [15]	4B	2,560	768	256	40	64	32

MLA MatShrink application order. MatShrink can be applied to MLA attention weights in the following four-step sequence, where each step is a valid application of Proposition 2 or the Q-K invariance result above:

1. Apply MatShrink to the per-head V-O pairs $(W_{\text{UV,i}}, W_{\text{O,i}})$: eliminates $h \cdot d_{\text{NOPE}}^2$ weights.
2. Apply the Q-K dot-product invariance to the NoPE portion of the per-head Q-K pairs: eliminates $h \cdot d_{\text{NOPE}}^2$ weights while preserving attention logits.

3. Apply MatShrink to the Q-latent pair $(W_{\text{DQ}}, W^*_{\text{UQ}})$, where W^*_{UQ} is the result of step 2. *Must follow step 2.*
4. Apply MatShrink to the KV-latent pair $(W_{\text{DKV}}, [W_{\text{UK}}; W^*_{\text{UV}}])$, where W^*_{UV} is the result of step 1. *Must follow step 1.*

Steps 1 and 2 act on disjoint weight pairs and are independent. After step 2, W_{UQ} is modified to W^*_{UQ}, but the composite map $W_{\text{DQ}}W^*_{\text{UQ}}$ is identical to $W_{\text{DQ}}W_{\text{UQ}}$ (by the Q-K dot-product invariance above); hence step 3 is a valid application of Proposition 2 to $(W_{\text{DQ}}, W^*_{\text{UQ}})$. Step 4 is analogous.

However, many MLA models deploy latent normalization by having an RMSNorm layer in-between each pair of down- and up-projections. This normalization prevents applying MatShrink to the Q-latent pair and KV-latent pair (steps 3 and 4).

6 MatShrink Combined with SVD-Based Compression

Composability with low-rank approximation. Let $W \approx \hat{W}_A\hat{W}_B$ be a rank-r approximation of $W \in \mathbb{R}^{d\times e}$ (e.g., from truncated SVD). Since Proposition 2 requires only that $\hat{W}_A \in \mathbb{R}^{d\times r}$, $\hat{W}_B \in \mathbb{R}^{r\times e}$, and $\operatorname{rank}(\hat{W}_B) = r$—conditions satisfied by any full-rank SVD truncation—MatShrink applies directly to $(\hat{W}_A, \hat{W}_B)$, eliminating an additional r^2 parameters while computing $\hat{W}_A\hat{W}_B$ exactly and leaving the approximation gap $\|W - \hat{W}_A\hat{W}_B\|$ unchanged.

7 Numerical Stability: Pivoted QR Basis Selection

MatShrink requires inverting a chosen $r \times r$ sub-block M of W_B; any invertible choice gives the same mathematical result. In practice, however, the choice matters when weights are stored in bf16 or lower precision: if M is nearly singular, M^{-1} contains very large entries that amplify bf16 rounding errors in the compressed weights. With the naive choice of the first r columns of W_B, this can cause perplexity to diverge (Table 5). Under fp32 the problem does not arise.

Fix: pivoted QR column selection. We select M using pivoted QR decomposition, which greedily picks the r columns of W_B that carry the most independent information—producing an M that is as far from singular as possible:

$$(Q, R, P) \leftarrow \text{pQR}(W_B), \qquad M \leftarrow W_B[:, P[:r]].$$

In practice this is a one-liner: `scipy.linalg.qr(W_B.T, pivoting=True)`. As Table 5 shows, this reduces bf16 regressions to within ± 0.05 perplexity points of the source model's bf16 baseline across all four tested models.

8 Experiments

We validate MatShrink's core claim of mathematical equivalence empirically on four models spanning both attention variants covered by the paper and two head dimension: SmolLM2-135M [16], a 30-layer decoder-only transformer with GQA ($n_{\text{heads}} = 9$, $n_{\text{KV}} = 3$, $d_{\text{head}} = 64$); GPT-2-124M [17], a 12-layer MHA transformer ($n_{\text{heads}} = 12$, $d_{\text{head}} = 64$); and Gemma-3 [18] at the 1B (text-only) and 4B (multimodal, text-backbone-only) sizes, both with $d_{\text{head}} = 256$, the regime where MatShrink's per-head r^2 savings accumulate most (Table 1). All experiments run on a single NVIDIA A100 GPU and evaluate perplexity on the full WikiText-2 raw test split using non-overlapping context-length windows.

8.1 Correctness under fp32

We apply V-O MatShrink to every attention layer. For each KV group (GQA) or each attention head (MHA), we select the transformation matrix M and compute M^{-1} in double precision. Reconstruction error at the identity block is $\mathcal{O}(10^{-14})$ on every model (fp64 machine precision).

With shrunk weights stored in fp32, baseline and MatShrink perplexities match exactly on all four models (Table 3), confirming that the transformation is mathematically equivalent and that inversion-induced numerical error is negligible at single precision.

Table 3: WikiText-2 perplexity before and after V-O MatShrink, fp32 storage, full test split.

Model	Source	MatShrink	Δ
GPT-2-124M (MHA)	29.9394	29.9394	0.0000
SmolLM2-135M (GQA)	15.4629	15.4629	0.0000
Gemma-3-1B (GQA)	14.1839	14.1838	-0.0001
Gemma-3-4B (GQA)	10.7664	10.7664	0.0000

8.2 Numerical precision at bf16 and the role of basis choice on the shared inner dimension

Problem. The precision issue is specific to bf16 storage. Under fp16 the identity-M variant is already essentially lossless (the fp16 rows of Table 5 show absolute regressions of only $+0.01$ ppl on SmolLM2-135M and $+0.06$ ppl on GPT-2-124M, both within benchmark noise); fp16's 10-bit mantissa carries enough precision to absorb the rewritten-weight magnitudes. bf16 trades 3 mantissa bits (10→7) for 3 additional exponent bits (5→8); it is the reduced mantissa that bites here. With the naive choice of M (the leading $r \times r$ sub-block of the target W_O slice), bf16 perplexity degrades substantially, and the degradation grows with head dimension: on $d_{\text{head}} = 64$ models the absolute ppl regression is $+1.51$ on SmolLM2-135M and $+18.94$ on GPT-2-124M; on $d_{\text{head}} = 256$ Gemma 3 it is $+25.29$ on the 1B variant (a qualitatively broken model) and $+8.98$ on the 4B variant. The root cause is that M^{-1} amplifies bf16 rounding error when M is ill-conditioned: the condition number of the naive M has a heavy tail on every model (up to $\kappa = 1.67 \times 10^5$ on GPT-2, $\kappa = 1.58 \times 10^5$ on Gemma-3-4B), and the entries of $W_V^* = W_V M$ and $W_{O_2}^* = M^{-1} W_{O_2}$ then span a range that bf16's 8-bit mantissa cannot faithfully represent. Error compounds across layers.

Fix: change of basis on the shared inner dimension, realized by pivoted QR. The two back-to-back matrices W_V and W_O meet along a shared r-dimensional axis (the head dimension, which is the inner dimension of the composite product). Applying any invertible change of basis $T \in \mathbb{R}^{r \times r}$ to this shared axis as $W_V \mapsto W_V T$ and $W_O \mapsto T^{-1} W_O$ leaves the composite $W_V W_O$ unchanged, so the model's output at every token position is preserved exactly. What T does change is *which* $r \times r$ sub-block of W_O ends up in MatShrink's leading position and therefore gets inverted as M. A pure permutation of this shared axis is the orthogonal special case, and because orthogonal transformations preserve singular values it leaves $\text{cond}(M)$ unchanged; a general invertible T reduces $\text{cond}(M)$ by one to three orders of magnitude on the models we test.

We select the change of basis implicitly by picking M directly as the best-conditioned $r \times r$ sub-block of W_O, identified by rank-revealing (pivoted) QR decomposition of W_O:

$$(Q, R, P) \leftarrow \text{QR}(W_O,\ \text{pivoting} = \texttt{True}), \qquad M \leftarrow W_O\big[:,\ P[:r]\big],$$

which, by classical argument, selects the r columns of W_O forming the best-conditioned leading block available. The corresponding explicit change of basis would be $T = M \cdot M_0^{-1}$ with $M_0 = W_O[:, :r]$ the naive leading sub-block; running vanilla MatShrink on $(W_V T, T^{-1} W_O)$ produces the same W_V^* and W_O^* as picking M directly, and the direct approach is the one-line shortcut used in the implementation. Alternative choices (column-L_2 sort, random sub-block sampling) are valid changes of basis as well and give partial improvements; pivoted QR is used because its conditioning guarantee is the strongest and because it is a one-line call in `scipy.linalg`. The resulting model is mathematically identical to the naive variant at fp32, stores the same number of weights, and loads in stock HuggingFace Transformers and vLLM without modification; the identity block simply lives at the column indices $P[:r]$ rather than at the contiguous prefix $[0, r)$.

Result. Pivoted QR reduces condition numbers by 1–3 orders of magnitude at the tail (Table 4) and collapses the bf16 perplexity regression to within ± 0.05 of the source model's bf16 perplexity on every tested configuration (Table 5). The fp32 perplexities are unchanged, as required by the mathematical invariance of the transformation.

8.3 vLLM compatibility

Because MatShrink in explicit-identity mode produces a standard checkpoint (identical tensor shapes and key names as the source), the shrunk model loads in vLLM [19] without modification to the runtime or model code. We verify end-to-end inference on SmolLM2-135M by running deterministic greedy generation before and after MatShrink; both produce coherent continuations from held-out

Table 4: Distribution of cond(M) across all (layer, head/KV-group) pairs. Pivoted QR reduces the tail by roughly two orders of magnitude at $d_{\text{head}} = 64$ and by two to three orders at $d_{\text{head}} = 256$, where the naive tail is largest.

Model	Strategy	median κ	95th pct. κ	max κ
GPT-2-124M (d_{head}=64)	identity	255	3.4×10^3	1.67×10^5
	pivoted	37	118	404
SmolLM2-135M (d_{head}=64)	identity	436	4.9×10^3	9.87×10^3
	pivoted	26	86	104
Gemma-3-1B (d_{head}=256)	identity	4.4×10^3	1.7×10^4	9.91×10^4
	pivoted	218	563	702
Gemma-3-4B (d_{head}=256)	identity	2.6×10^3	2.4×10^4	1.58×10^5
	pivoted	156	570	5.4×10^3

Table 5: WikiText-2 perplexity on full test split, with pivoted-QR M-selection vs. naive identity-slice M-selection. fp32 is mathematically invariant (identical to four decimal places). Under bf16, pivoted-QR closes the gap to the source model from as large as $+25.29$ absolute (Gemma-3-1B identity-M) to within ± 0.05 on every model. Gemma 3 does not support fp16 inference on the source model (dynamic-range overflow), so only fp32 and bf16 are reported for those rows.

Model	Precision	Source	Identity-M	Pivoted-M
GPT-2-124M	fp32	29.9394	29.9394	29.9394
	fp16	29.9482	30.0038	29.9467
	bf16	30.2090	49.1541	**30.1928**
SmolLM2-135M	fp32	15.4629	15.4629	15.4629
	fp16	15.4634	15.4775	15.4634
	bf16	15.4750	16.9885	**15.4787**
Gemma-3-1B	fp32	14.1839	14.1838	14.1839
	bf16	14.1932	39.4825	**14.1520**
Gemma-3-4B	fp32	10.7664	10.7664	10.7664
	bf16	10.7877	19.7706	**10.8055**

prompts. The pivoted-QR variant is equivalent in this respect: the identity block lives at non-contiguous column indices, but every tensor retains its original shape and name, so no loader changes are required.

Because the identity blocks are stored as dense tensors, the runtime performs redundant matmuls and throughput is unchanged; realizing the compute savings implied by the identity structure requires runtime support, which we leave to future work.

Note on further quantization. A separate consequence of pivoted-QR selection is that W_V^* and W_O^* retain the magnitude range of the source W_V and W_O (since a well-conditioned M^{-1} does not inflate entries), whereas the naive identity-M variant produces rewritten weights with inflated magnitudes that track $\kappa(M)$. Downstream 8-bit or 4-bit weight quantization, which depends on a bounded weight distribution, should therefore compose more gracefully with pivoted-QR MatShrink than with the identity variant. Systematic evaluation of MatShrink plus quantization is left to future work.

9 Conclusion

We presented MatShrink, a theoretically grounded, empirically validated framework for losslessly eliminating r^2 parameters from any pair of consecutive linear layers. The core result (Proposition 2) is an elementary change-of-basis identity; the pivoted QR analysis (Section 7) establishes the numerically optimal choice of sub-block. Specializations to MHA, MLA, GQA, MQA (Sections 3–4) quantify savings of 8–17% of attention parameters for MHA. Experiments on four models confirm exact fp32 equivalence and near-exact bf16 equivalence, with compressed checkpoints loading in production inference engines without modification.

Limitations and future work. The weight savings currently do not yield proportional throughput improvements, since identity blocks are stored and multiplied as dense tensors; realizing the compute

reduction requires structured-sparse runtime support. When pivoted QR is used, the column permutation must also be stored and applied at inference time (or folded into subsequent layers if possible); finding compact representations of this permutation and efficient kernel-level implementations is an additional open problem. Beyond post-training compression, future models could adopt MatShrink's reduced weight structures from the outset, training directly on the smaller parameterization and achieving better parameter efficiency from scratch. Extending empirical validation to MLA models, evaluating composition with post-training quantization (Section 7), integrating MatShrink-aware kernels into production engines (llama.cpp [20], SGLang [21]) are natural directions for future work.

References

[1] OpenMachine. Transformer tricks. 2024. URL https://github.com/OpenMachine-ai/transformer-tricks.

[2] Chi-Heng Lin, Shangqian Gao, James Seale Smith, Abhishek Patel, Shikhar Tuli, Yilin Shen, Hongxia Jin, and Yen-Chang Hsu. MoDeGPT: Modular Decomposition for Large Language Model Compression. 2024. *arXiv:2408.09632*.

[3] Pratyusha Sharma, Jordan T Ash, and Dipendra Misra. The truth is in there: Improving reasoning in language models with LAyer-SElective Rank reduction. December 2023. *arXiv:2312.13558*.

[4] Nils Graef and Andrew Wasielewski. Slim attention: cut your context memory in half without loss – K-cache is all you need for MHA. March 2025. *arXiv:2503.05840*.

[5] Nils Graef. KV-weights are all you need for skipless transformers. April 2024. *arXiv:2404.12362*.

[6] Marko Karbevski and Antonij Mijoski. Key and Value weights are probably all you need: On the necessity of the Query, Key, Value weight triplet in encoder-only and decoder-only transformers. January 2026. *arXiv:2510.23912*.

[7] Kaichao You, Guo Qin, Anchang Bao, Meng Cao, Ping Huang, Jiulong Shan, and Mingsheng Long. Efficient ConvBN blocks for transfer learning and beyond. May 2023. *arXiv:2305.11624*.

[8] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. November 2022. *arXiv:2211.10438*.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. June 2017. *arXiv:1706.03762*.

[10] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. December 2022. *arXiv:2212.04356*.

[11] CodeGemma Team, Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A Choquette-Choo, Jingyue Shen, Joe Kelley, Kshitij Bansal, Luke Vilnis, Mateo Wirth, Paul Michel, Peter Choy, Pratik Joshi, Ravin Kumar, Sarmad Hashmi, Shubham Agrawal, Zhitao Gong, Jane Fine, Tris Warkentin, Ale Jakse Hartman, Bin Ni, Kathy Korevec, Kelly Schaefer, and Scott Huffman. CodeGemma: Open Code Models Based on Gemma. June 2024. *arXiv:2406.11409*.

[12] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. October 2019. *arXiv:1910.10683*.

[13] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. May 2023. *arXiv:2305.13245*.

[14] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Yang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jin Chen, Jingyang Yuan, Junjie Qiu, et al. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. *arXiv:2405.04434*.

[15] Yuan Yao, Tianyu Yu, Ao Zhang, Chongyi Wang, Junbo Cui, Hongji Zhu, Tianchi Cai, Haoyu Li, Weilin Zhao, Zhihui He, Qianyu Chen, Huarong Zhou, Zhensheng Zou, Haoye Zhang, Shengding Hu, Zhi Zheng, Jie Zhou, Jie Cai, Xu Han, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. MiniCPM-V: A GPT-4V Level MLLM on Your Phone. 2024. *arXiv:2408.01800*.

[16] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Lewis Tunstall, Agustín Piqueres, Andres Marafioti, Cyril Zakka, Leandro von Werra, and Thomas Wolf. SmolLM2 - with great data, comes great performance, 2024. URL https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B.

[17] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. 2019.

[18] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean-Bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, et al. Gemma 3 Technical Report. March 2025. *arXiv:2503.19786*.

[19] vLLM Project. vLLM. URL https://github.com/vllm-project/vllm.

[20] Georgi Gerganov. llama.cpp. URL https://github.com/ggml-org/llama.cpp.

[21] SGLang. SGLang. URL https://github.com/sgl-project/sglang.